



Système d'Exploitation 2

Chapitre 2 : La gestion de la mémoire (partie 4)

Dr. M. Baba Ahmed

Plan

- ❑ Le défaut de page
- ❑ Le chargement de page
- ❑ La pagination a la demande
- ❑ Remplacement de pages
- ❑ Les algorithmes de remplacement de pages
 - L'algorithme de Belady (Optimal)
 - L'algorithme FIFO
 - L'algorithme LRU

Plan

- L'algorithme FINUFO
- L'algorithme NRU
- Cache de table de page (Translation Lookaside Buffer) (TLB)
 - Mémoire associative
 - Recherche associative dans TLB
 - L'utilisation de TLB
 - Temps d'accès mémoire réel avec TLB
 - Comptage d'utilisation de TLB

Le défaut de page

- ❑ Lorsqu'un pointeur tente d'accéder à une page qui n'est actuellement pas chargée en mémoire physique, une erreur se produit (Défaut de page)
 - C'est une **occurrence normale** et non observée par l'utilisateur.
- ❑ Le système de gestion de la mémoire alloue un cadre à la page demandée et charge la page en MC (Swap-in)
 - S'il n'y a pas de cadre libre → Remplacement de page

Le chargement de pages

- ❑ Lors d'un défaut de page, un signal est envoyé au processeur qui redonne la main au SE pour que celui-ci s'occupe de la page manquante.
- ❑ L'objectif du SE est de ne pas transférer une seule page mais d'essayer de transférer les pages qui ont le plus de chance d'être utile.

La pagination a la demande

- ❑ Selon le concept de mémoire virtuelle, l'exécution d'un processus, nécessite que seules une partie du processus doit être présente dans la mémoire principale,
- ❑ Décider quelles pages doivent être conservées dans la mémoire principale et lesquelles doivent être conservées dans la mémoire secondaire va être difficile car il est impossible de prédire à l'avance quelles pages sont nécessaire pour un processus.

Solution :

- ▶ Pagination a la demande.

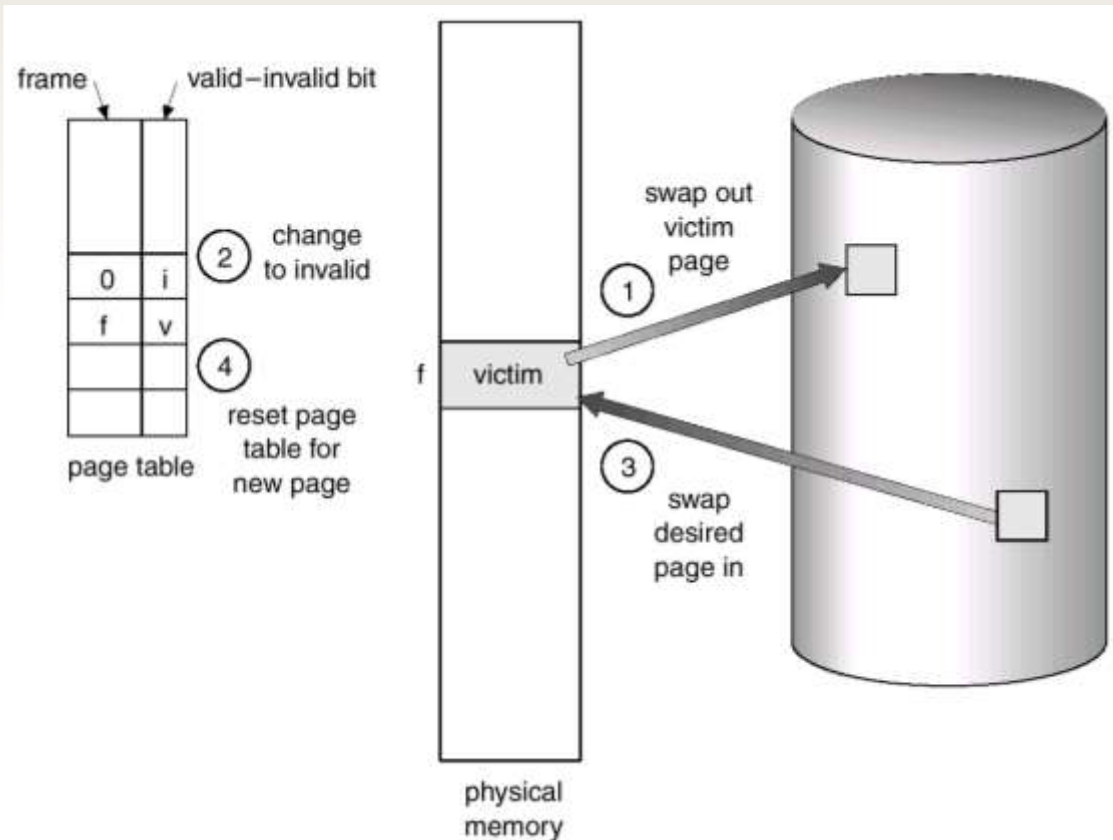
La pagination a la demande

- ❑ Il suggère de conserver toutes les pages des cadres dans la mémoire secondaire jusqu'à ce qu'elles soient requises.
- ❑ Il est indiqué de ne charger aucune page dans la mémoire principale tant qu'elle n'est pas requise.
- ❑ Chaque fois qu'une page est référencée pour la première fois dans la mémoire principale, cette page sera trouvée dans la mémoire secondaire.

Remplacement de pages

1. Trouver la page demandée sur disque
2. Trouver un cadre de page libre :
 - Si un cadre de page libre existe, l'utiliser
 - Si un cadre de page libre n'existe pas, utiliser un algorithme de remplacement de page pour choisir un cadre de page **victime**
3. Chargé la page demandée dans le cadre de page libre. Mettre à jour les tables des pages et des cadre de page.
4. Relancer le processus

Remplacement de pages



Remplacement de pages

❑ Bit de modification (dirty bit) :

- La 'victime' doit-elle être réécrite en mémoire secondaire?
 - Seulement si elle a été changée depuis qu'elle a été amenée en mémoire principale
sinon, sa copie sur disque est encore fidèle
- ❑ Bit de modification de chaque descripteur de page indique si la page a été changée

Les algorithmes de remplacement de pages

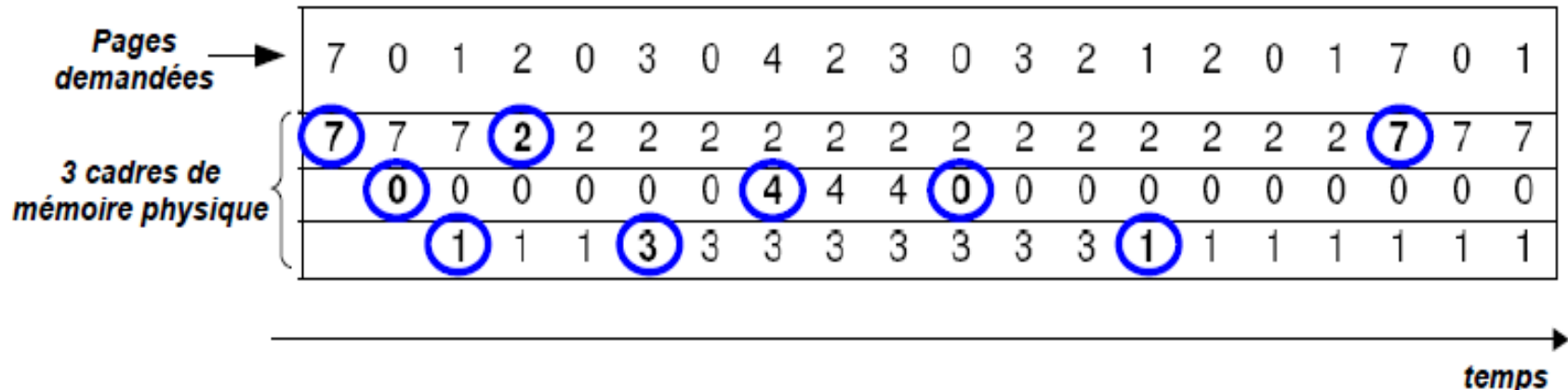
- ❑ Choisir la page à retirer de manière à **minimiser le nombre** de défauts de page
 - Choisir la victime de façon à minimiser le taux de défaut de pages pas évident!!!
Page dont nous n'aurons pas besoin dans le futur?
 - Page pas souvent utilisée?
 - Page qui a été déjà longtemps en mémoire??
- ❑ **Les algorithmes de choix de pages** à remplacer doivent être conçus de façon à minimiser le taux de défaut de pages à long terme

Les algorithmes de remplacement de pages

- ☐ Algorithme de Belady (optimal)
- ☐ Algorithme FIFO (First In First Out)
- ☐ Algorithme LRU (Least Recently Used)
- ☐ Algorithme FINUFO (First In Not Used First Out)
- ☐ Algorithme NRU (Not Recently Used)

L'algorithme de Belady (Optimal)

- ❑ Principe : choisir la page qui sera demandée le plus tard possible dans le futur.
 - La page non réutilisée ou qui sera réutilisée le plus loin dans le futur
- ❑ Stratégie théorique et impossible à mettre en œuvre dans la réalité

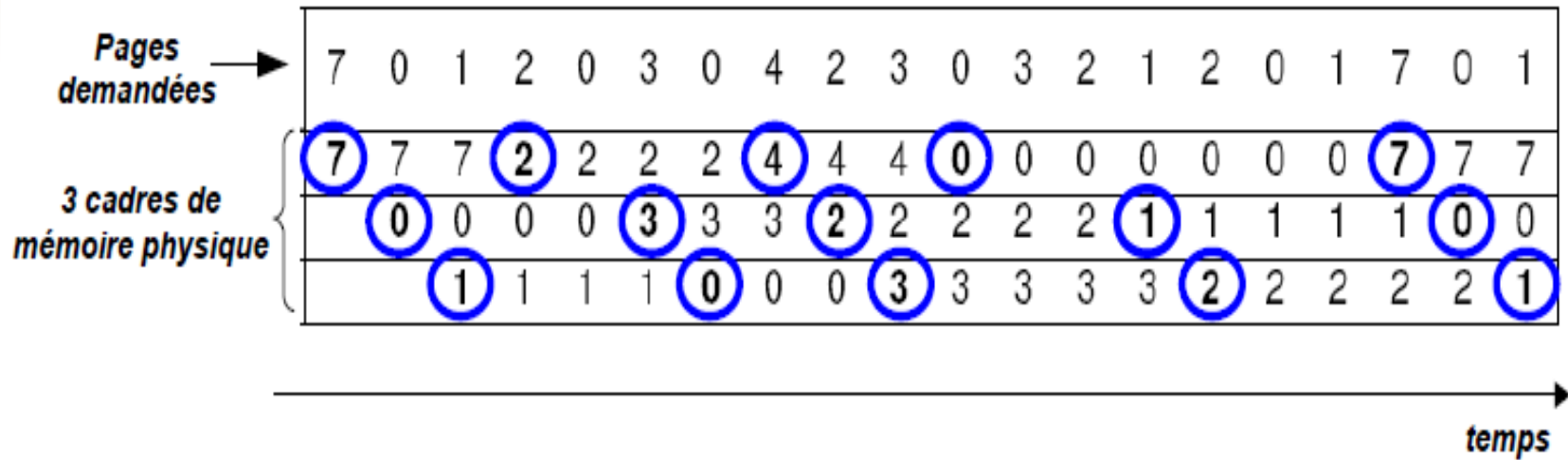


L'algorithme de Belady (Optimal)

- ❑ Cette approche entraîne le moins de défauts de page.
- ❑ Cependant, cette approche n'est pas réalisable, car nous ne pouvons jamais vraiment savoir quelle page a le plus de temps pour être remplacée.
- ❑ Cet algorithme n'est que théorie, il n'est pas implémentable.

L'algorithme FIFO (First In First Out)

- ❑ Choisir la page la plus ancienne en mémoire.
- ❑ La page la plus ancienne est la victime (celle qui a été placée en mémoire depuis le plus long temps).



L'algorithme FIFO (First In First Out) (Anomalie)

- ❑ Stratégie qui ne tient pas compte de l'utilisation de chaque page
- ❑ Algorithme rarement utilisé car il génère **beaucoup de défauts de page**
- ❑ L'augmentation du nombre de cadres peut se traduire par un **incrément** du nombre de défauts de page avec l'algo FIFO.

**Algorithme FIFO
avec 3 cadres de pages**

	0	1	2	3	0	1	4	0	1	2	3	4
	0	0	0	3	3	3	4	4	4	4	4	4
		1	1	1	0	0	0	0	0	2	2	2
			2	2	2	1	1	1	1	1	3	3

9 défauts de page

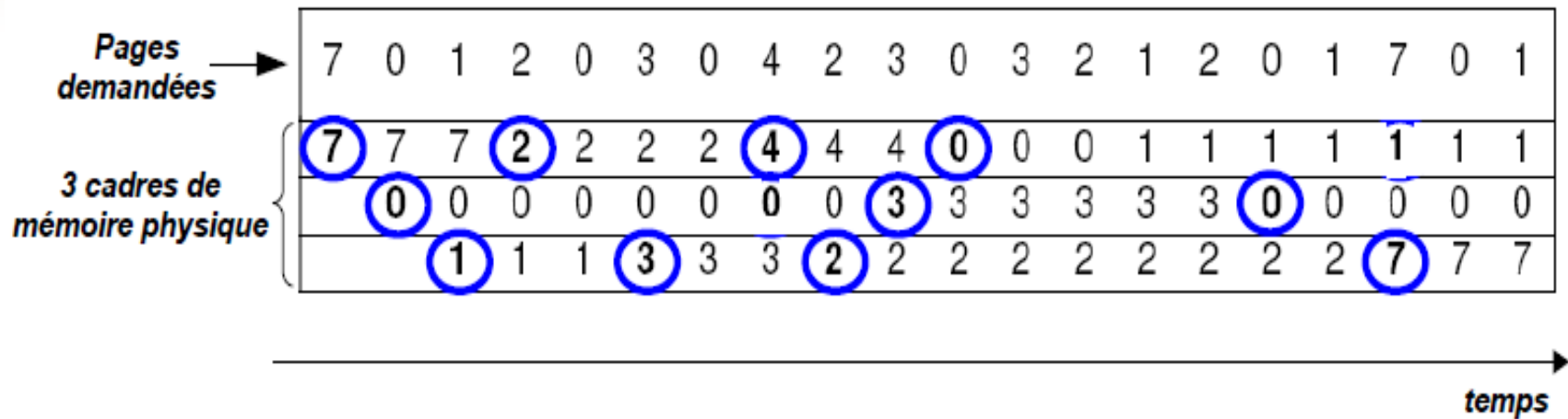
**Algorithme FIFO
avec 4 cadres de pages**

	0	1	2	3	0	1	4	0	1	2	3	4
	0	0	0	0	0	0	4	4	4	4	3	3
		1	1	1	1	1	1	0	0	0	0	4
			2	2	2	2	2	2	1	1	1	1
				3	3	3	3	3	3	2	2	2

10 défauts de page !

L'algorithme LRU (Least Recently Used)

- ❑ Choisir la page la moins récemment utilisée
- ❑ La page sortante est celle qui n'a plus été utilisée depuis le plus longtemps.



L'algorithme LRU (Least Recently Used)

- ❑ Stratégie plus complexe a implémenter
- ❑ Nécessite un tri dynamique des pages par ordre d'accès a chaque référence mémoire
- ❑ Algorithme couteux mais efficace

Exemple:

Les mêmes références de pages avec 4 cadres de mémoire physique.

L'algorithme FINUFO (First In Not Used First Out)

- ❑ L'algorithme de la **seconde chance**
- ❑ **Principe** : algorithme FIFO + examen du bit de référence (R)
 - Le bit R associé à chaque page est positionné à 1 à chaque fois qu'une page est réutilisée par un processus.
 1. bit de référence a 0 : choix de la page (FIFO)
 2. bit de référence a 1 : on donne **une seconde chance** a la page (la page à été utilisée récemment). Ensuite, on repositionne R à 0 et on cherche une autre victime.

L'algorithme FINUFO (First In Not Used First Out)

0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4

0(o)	0(o)	0(o)	0(o)	2(o)	2(o)	2(o)	2(o)	2(1)	2(1)	2(o)	2(o)	1(o)	1(o)	1(o)	1(o)	3(o)	3(o)
	4(o)	4(o)	4(1)	4(1)	4(1)	4(o)	4(1)	4(1)	4(1)	4(o)	4(1)	4(o)	4(1)	4(o)	4(1)	4(o)	4(1)
		1(o)	1(o)	1(o)	1(o)	3(o)	3(o)	3(o)	3(o)	0(o)	0(o)	0(o)	0(o)	2(o)	2(o)	2(o)	2(o)
D	D	D		D		D				D		D		D		D	

L'algorithme NRU (Not Recently Used)

- ❑ Afin de permettre au Système d'exploitation de collecter des statistiques utiles sur les pages utilisées et celles qui ne le sont pas, deux bits sont utilisés :
 1. Le bit **R** a 1 lorsque la page est référencée (lecture ou écriture).
 2. Le bit **M** a 1 lorsque la page est écrite (c'est-à-dire modifiée).
 3. Le bit **R** a 0 lorsque la page n'a pas été référencée
 4. Le bit **M** a 0 lorsque la page n'a pas été modifiée (écrite)

L'algorithme NRU (Not Recently Used)

- ❑ Lorsqu'un défaut de page se produit, le système d'exploitation inspecte toutes les pages et les divise en quatre catégories en fonction des valeurs actuelles de leurs bits R et M :

Classe 0	Non référencée	Non modifiée
Classe 1	Non référencée	Modifiée
Classe 2	Référencée	Non modifiée
Classe 3	Référencée	Modifiée

L'algorithme NRU (Not Recently Used)

- ❑ L'algorithme NRU supprime une page au hasard de la classe non vide numérotée la plus basse, exemple :
 - La page 0 est de classe 2
 - La page 1 est de classe 1
 - La page 2 est de classe 0
 - La page 3 est de classe 3

La page 2 fait partie de la classe la plus basse, et donc doit être remplacée par NRU

Cache de table des pages (Translation Lookaside Buffer (TLB))

- ❑ **(TLB)** est un type de cache mémoire qui stocke les traductions récentes de la mémoire virtuelle vers des adresses physiques pour permettre une récupération plus rapide.
- ❑ Ce cache à grande vitesse est configuré pour garder une trace des entrées de table de pages (PTE) récemment utilisées.
- ❑ A chaque instruction, la MMU parcourt tout le TLB pour faire la conversion d'adresse.
 - En cas de défaut de TLB : le système met à jour le TLB

Mémoire associative

- ❑ Mémoire adressable par contenu (CAM) « Content Adressable Memory »
- ❑ Afin de récupérer les données résidant sur la mémoire physique, le système d'exploitation fournit l'adresse mémoire où les données sont stockées.
- ❑ Les données stockées sur une mémoire (CAM), sont accessibles en recherchant le contenu lui-même, et la mémoire récupère les adresses où ce contenu peut être trouvé.
- ❑ Utilisée dans certaines applications de recherche à très grande vitesse.

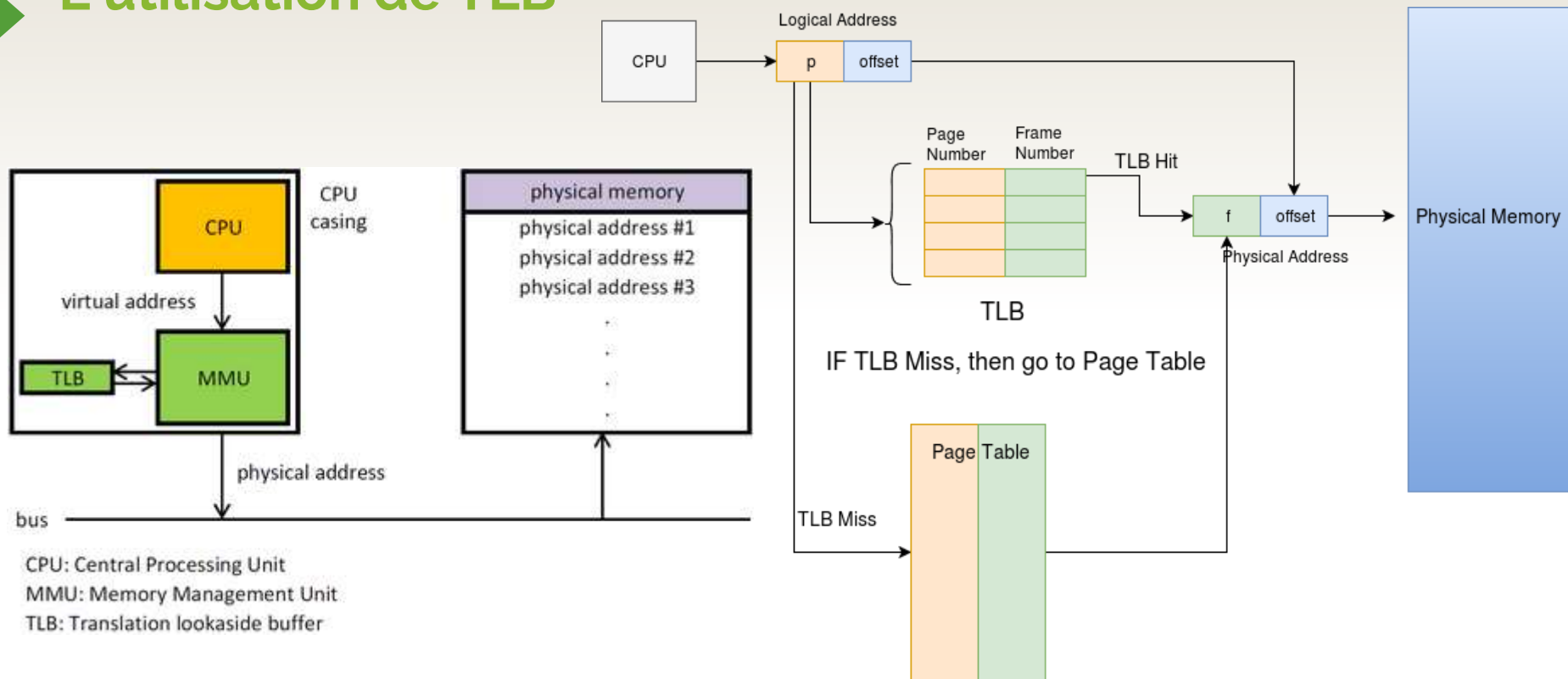
Recherche associative dans TLB

- ❑ Le TLB est parfois implémenté en tant que **mémoire adressable par le contenu** (mémoire associative).
- ❑ La clé de recherche dans une tel mémoire est **l'adresse virtuelle** et le résultat de la recherche est une **adresse physique**.
- ❑ Si l'adresse demandée est présente dans le TLB, la recherche donne rapidement une correspondance et l'adresse physique récupérée peut être utilisée pour accéder à la mémoire. C'est ce qu'on appelle **un hit TLB**.
- ❑ Si l'adresse demandée n'est pas dans le TLB, c'est un échec (**Miss**) et la traduction se poursuit en consultant la table des pages

L'utilisation de TLB

- ❑ Le TLB mémorise les derniers couples (page, cadre)
- ❑ Sur réception d'une adresse logique, le processeur examine le TLB
 - Si cette entrée de page y est , le numéro de cadre en est extrait
 - Sinon, le numéro de page indexe la table de page du processus (en mémoire)
 1. Cette nouvelle entrée de page est mise dans le TLB
 2. Elle remplace une autre, pas récemment utilisée
- ❑ Le contenu du TLB est remplacé quand le CPU change de processus

L'utilisation de TLB



Temps d'accès mémoire réel avec TLB

Le temps effectif d'accès/Effective memory access time, **TEA**= $(m + c) \alpha + (2m + c)(1 - \alpha)$

- ❑ **m**: temps d'accès à la mémoire physique (RAM)
- ❑ **c**: temps d'accès à la mémoire associative (TLB)
- ❑ **α** : probabilité de touche (hit)
 - Probabilité qu'un numéro de page soit trouvé dans TLB

Temps d'accès mémoire réel avec TLB

Le temps effectif d'accès/Effective memory access time, **TEA**= $(m + c) \alpha + (2m + c)(1 - \alpha)$

Exemple :

Supposant un temps d'accès à la mémoire 100 ns, et un temps d'accès à la mémoire associative (TLB) est 20 ns, Quel est le temps moyen pour un cache hit de 80% ?

Temps d'accès mémoire réel avec TLB

Solution :

$$TEA = 0,8 (20+100) + (1-0,8)*(20+2(100))$$

$$TEA = 0,8(120) + (0,2)(220) = 140 \text{ ns}$$

Comptage d'utilisation de TLB

- ❑ Chaque paire dans le TLB est associée à un compteur pour savoir si cette paire a été utilisée récemment.
 - La page non récemment utilisée est remplacée par la dernière paire dont on a pas besoin
- ❑ Exemple :

Compt	N°page	N°cadre
1	3	15
0	7	19
3	0	17
2	2	23